Undead Legion

Extended Edition

Physics · Navigation · Combat AI · Armies

Unity 6 (6000.4) · Universal Render Pipeline · AI Navigation

Contents

1. Overview
2. Requirements and setup
3. Folder structure
4. The extended prefabs
5. Ragdolls
6. Hit reactions
7. Health, damage and death
8. Cloth
9. Rise and dissolve effects, glowing eyes
10. NavMesh movement
11. Combat AI
12. Army spawner
13. The arena demo
14. The extended animation browser
15. Performance
16. Support

1. Overview

The Extended Edition contains the complete Undead Legion pack (characters, armour modules, weapons, animations and the browser demo, described in `UndeadLegion/Documentation/UndeadLegion_Documentation.pdf`) and adds the systems that turn the skeletons into a playable army. Everything new lives in `UndeadLegion/Extended`; the base assets are used as they are.

- **Full-body ragdolls**: 11 rigidbodies with joints on every character, used as hitboxes while animated, with an impulse on death and a blend back to the animation for revives and get-ups.
- **Hit reactions**: physical flinches layered over any clip, so a struck skeleton recoils while it keeps walking or attacking.
- **Cloth**: robes, skirts and pants simulate, pinned at the waist and colliding with the legs.
- **Rise and dissolve effects**: skeletons rise out of the ground and turn to dust, with a glowing edge. **Glowing eyes** in any colour.
- **NavMesh movement**: an agent drives directional locomotion blend trees built from the pack's measured walk, run, back and strafe clips.
- **Combat AI**: finds the nearest enemy, closes in and attacks with the moves that suit the equipped weapon: melee, arrows, spells and an area cast.
- **Army spawner**: formations of varied skeletons with mixed armour, suitable loadouts, tints and team colours.
- An **arena demo**: two armies, a battle, and a skeleton you can take over; and the **animation browser** with the extended characters, cloth, hits, ragdolls and the effects.

2. Requirements and setup

- Unity 6 (6000.4 or newer), **Universal Render Pipeline** 17.4.
- The **AI Navigation** package (`com.unity.ai.navigation`, 2.x) for the NavMesh. It is listed in the project manifest; if you import the package into another project, install it from the Package Manager.

Open `UndeadLegion/Extended/Scenes/UndeadLegion_Extended_Demo.unity` and press Play. To use the extended skeletons in your own scene, bake a NavMesh there (add a *NavMesh Surface* to your level and press *Bake*), then drag a prefab from `UndeadLegion/Extended/Prefabs` onto it, or add an `ArmySpawner`.

The extended prefabs are prefab variants of the base characters: changes to a base prefab (its armour set, its hand slots) carry over to its extended variant.

3. Folder structure

Folder	Contents
`Extended/Prefabs`	`PFX_Skeleton<Class>`: the six characters with physics, navigation, AI and effects
`Extended/Animations`	`AC_Skeleton_Extended`: the base controller plus locomotion blend trees and upper-body weapon idles
`Extended/Scripts`	the runtime components (section 4 onwards), namespace `UndeadLegion.Extended`
`Extended/Shaders`	`UndeadLegion/Lit Dissolve`: URP Lit with a height cut, noise and a glowing edge
`Extended/Materials`	the arena's stone material and post-processing profile
`Extended/Scenes`	the arena demo and the extended animation browser, each with its baked NavMesh
`Extended/Documentation`	this document

4. The extended prefabs

Each `PFX_Skeleton<Class>` is the base character with `AC_Skeleton_Extended` and these components added:

Component	Role
`SkeletonRagdoll`	11 bodies (hips, chest, head, upper and lower arms and legs) with joints; kinematic hitboxes while animated
`SkeletonHitReaction`	physical flinches over the animation
`SkeletonHealth`	health, team, damage events, the death mode
`SkeletonCloth`	cloth on robes, skirts and pants
`SkeletonDissolve`	rise and dissolve effects
`NavMeshAgent` + `SkeletonNavController`	movement on the NavMesh (radius 0.32 m, height 1.8 m)
`SkeletonAI`	the combat behaviour; disable it to drive the skeleton yourself

The base components stay: `SkeletonModules` (armour), `SkeletonWeapon` (loadouts), `SkeletonTwitch` and `SkeletonFootLock`. Every component is independent: remove the ones you do not need (for example the AI in a player character, or the cloth in a large crowd).

5. Ragdolls

While the character animates, the ragdoll bodies are kinematic and follow the bones, so their colliders work as **hitboxes** (a raycast or a trigger tells you which body part was struck). `Activate(impulse, point)` hands the body to physics: the Animator and the root capsule switch off, every body keeps the velocity it had in the animation, the impulse is applied to the body nearest the point, and held weapons drop with their own physics (`dropWeapons`). `BlendToAnimation()` moves the character under the fallen hips and blends the pose back to the Animator over

`blendTime` seconds, for revives and get-ups. The bodies are generated by `Build()` from the humanoid bones; `totalMass` (35 kg) is spread by the usual ragdoll proportions. A body's parts never collide with each other. `angularDamping` (0.6) and `headAngularDamping` (5) settle a fallen body: an unsupported head comes to rest instead of swinging on its joint.

```
var ragdoll = skeleton.GetComponent<SkeletonRagdoll>();
ragdoll.Activate(hitDirection * 80f, hitPoint);     // fall with a push
ragdoll.BlendToAnimation();                         // stand back up on the animation
```

6. Hit reactions

`SkeletonHitReaction.Hit(point, impulse)` kicks damped angular springs on the struck bone and, with `falloff`, on its parents up to the hips (spine, chest, neck, head, shoulders and arms react). The springs are applied after the Animator every frame, so a hit reads on top of whatever plays and settles back by itself. `stiffness` sets how fast a bone returns, `damping` how much it wobbles, `strength` how far it moves per unit of impulse, `maxAngle` the largest offset. `SkeletonHealth` calls it on every hit, so you rarely need to call it directly.

7. Health, damage and death

```
skeleton.GetComponent<SkeletonHealth>().TakeDamage(25f, hitPoint, hitDirection * 60f, attacker);
```

Setting	Effect
`maxHealth`, `health`	hit points
`team`	skeletons of the same team never target each other
`deathMode`	**Ragdoll** (fall with the killing blow's impulse), **DeathClip** (one of the four death animations), **DeathClipThenRagdoll** (the clip starts, the ragdoll takes over after `ragdollAfter` seconds)
`hitClips`	additive reactions (`Hit_01`, `Hit_02`), one picked at random and played on the Animator's Hit layer on every hit that does not kill, over whatever the skeleton is doing, together with the physical flinch
`dissolveAfter`	seconds after death before the corpse turns to dust (0 = stays); `destroyWhenDissolved` removes it
`invulnerable`	hits still flinch, play the hit clips and raise `onDamaged`, but health never drops (a player character that cannot die)
`blocking`, `blockArc`	while blocking, a hit from within `blockArc` degrees of the front does no damage and gives only `blockedReaction` of the flinch
`onDamaged`, `onDied`	UnityEvents for your game logic (score, sounds, loot)

`Die()` kills at once, `Revive()` restores full health and blends a ragdoll back to the animation. `SkeletonHealth.All` lists every living and dead skeleton in the scene.

8. Cloth

`SkeletonCloth` adds Unity Cloth to the worn armour pieces named in its `rules` (Skirt, Robe and Pants by default). Each cloth is pinned at the waist and frees gradually towards the hem: `pinnedBand` metres below the waist stay fixed, the hem may move up to `maxDistance` from its skinned position. The legs' ragdoll capsules are the colliders, so a robe swings around the legs instead of through them. The Knight's set is plate (`rigidSets`): its pieces never simulate, also when another skeleton wears them. Call `Refresh()` after changing the worn armour, and `SetSimulate(false)` to switch cloth off. Beyond `cullDistance` from the main camera the cloth stops simulating.

9. Rise and dissolve effects, glowing eyes

`SkeletonDissolve.DissolveIn()` raises a skeleton out of the ground; `DissolveOut(destroyAfter)` turns it to dust from the top down. For the duration of an effect each renderer (body, armour, held and dropped weapons) uses a copy of its material on the `UndeadLegion/Lit Dissolve` shader, a URP Lit shader with a cut height, noise and a glowing band; the original materials come back when a rise ends, and per-renderer colours (tints) are kept. Set `duration`, `edgeColor` (HDR), `edgeWidth` and `noiseScale` to taste; `dissolveInOnEnable` plays the rise whenever the object is enabled (useful with object pools).

`SkeletonEyes` (a base-pack script, on every character prefab) places two small glowing spheres in the eye sockets and follows the head. The colour is HDR: values above 1 glow with Bloom (the arena uses URP's Bloom post effect). `SetColor()` changes it at runtime; `addLight` adds a faint light at the face, off by default because every light costs in a crowd.

10. NavMesh movement

`SkeletonNavController` reads the `NavMeshAgent`'s velocity into the controller's `MoveX` / `MoveZ` parameters (metres per second in the character's frame) and plays one of two 2D directional blend trees on the Base layer:

Gait	State	Clips (walk / run / back / strafe)	Walk / run speed
Heavy	`Locomotion_Heavy`	the shambling `_01` set	0.52 / 1.14 m/s
Upright	`Locomotion_Upright`	the upright `_02` set	1.28 / 1.87 m/s

The agent plans the path and steers; the clips' own **root motion** moves and turns the body (the agent follows it and keeps it on the NavMesh), so the feet plant and the body turns with each step as authored. The blend-tree positions are the clips' measured speeds, so the wanted speed picks the matching clip. The agent's speed is set from the gait (walk or `run`). When the skeleton stops, it plays the idle that suits its weapon (`Idle_1H_Combat`, `Idle_TwoHanded`, `Idle_Bow`, `Idle_Staff`). While it moves, the walk or run animates the whole body, arm swing included; only the idles listed in `upperHoldWhileMoving` (by default `Idle_TwoHanded`, so both hands stay on a two-hander's shaft) are held on the UpperBody layer.

```
var nav = skeleton.GetComponent<SkeletonNavController>();
nav.MoveTo(destination, running: true);
nav.FaceTowards(enemy.position);
nav.PlayAction("Taunt_01");                // a one-shot; a step in the clip moves the agent
nav.OnLoadoutChanged();                    // after equipping another weapon
```

11. Combat AI

`SkeletonAI` picks the nearest living enemy within `sightRange` (or your `target`), runs toward it and slows to a walk `walkWithin` metres (0.8) outside its attack's reach (archers keep walking), stops at its weapon's **engage distance**, turns to face it and attacks, waiting `attackCooldown` seconds between attacks. A melee weapon's engage distance is its modules' `engageRange` (centre to centre, see the table; `meleeRange`, 1.1 m, when a module sets none); a bow, wand or staff keeps the range of its shots. Once engaged, it keeps attacking until the target is `engageSlack` (0.3 m) beyond that distance, and a skeleton blocked by others for `blockedAttackAfter` (0.4 s) within its weapon's reach strikes from where it stands. Damage lands at the moment in the clip where the blow connects. The attacks come from **attack modules** (`AttackModule`, the `attackModules` list on the component): each module names the weapon loadouts it attaches to, and a skeleton fights with the moves of every module that matches what it holds, picking one at random (never the same twice in a row when it has a choice). The defaults:

Module	Weapons	Attacks	Engage	Reach
Sword	sword, sword + shield	`Attack_R_01_Stab`, `Attack_R_02_Swing`, `Attack_R_03_Swing`	1.1 m	1.7–1.8 m
Dagger	dagger, two daggers	the same three	0.9 m	1.7–1.8 m
Dagger, left hand	two daggers	`Attack_L_01_Stab`, `Attack_L_02_Swing`, `Attack_L_03_Swing`	0.9 m	1.5–1.6 m
Axe / Axe, heavy	axe + shield	the three right-hand attacks / their `_Heavy` variants: a bigger lunge, more damage	1.2 m	1.7–1.9 m
Mace / Mace, heavy	mace	the two swings / their `_Heavy` variants (no stab)	1.2 m	1.7–1.8 m
Two-handed	longsword, battle axe	`Attack_2H_01_Swing`, `Attack_2H_02_Swing`	1.5 m	2.1–2.2 m
Two-handed, heavy	battle axe	`Attack_2H_01_Swing_Heavy`, `Attack_2H_02_Swing_Heavy`	1.5 m	2.2–2.3 m
Bow	recurve bow	`Shoot_01`: the arrow flies and hits after its flight time	16 m	16 m
Wand	wand	`Cast_Wand_01/02`: a homing bolt (`MagicBolt`)	11 m	11 m
Staff	staff	`Cast_Staff_01` bolt, and `AOE_Cast` hitting every enemy within 3 m of the target (8 s cooldown)	12 m	9–12 m
Unarmed	nothing held	light swings with both arms	0.8 m	1.4 m

So swords and daggers use regular attacks only, axes and maces mix regular and heavy ones (the mace only swings), and a second dagger adds the left-hand attacks. The weapon's pace comes from the base pack (the controller's `AttackSpeed`, set by `SkeletonWeapon`: sword 30 % faster, dagger 70 % faster), the same in every scene; a move's own `speed` and the component's `attackSpeed` multiply on top. A heavy module is skipped when `allowHeavy` is off, which it is on the assassin: it fights with regular attacks whatever it holds. With `allowArea` off a skeleton never uses the area cast (the player demo turns it off for a player necromancer). Add a module, or change one's weapons and moves, in the inspector to give a weapon its own set.

`damageScale` and `attackSpeed` tune difficulty; `wanderRadius` lets idle skeletons roam around their start point; `magicColor` colours the spells. Disable the component to take control of a skeleton (the arena demo does this when you select one).

12. Army spawner

`ArmySpawner` places `count` skeletons in a formation of `columns` × rows, `spacing` metres apart, snapped to the NavMesh and facing the spawner's forward. Each one gets:

- a random character from `characters`;
- a mixed armour set from `armourCatalogue`: each slot keeps the class's own piece or borrows another class's (`mixChance`), within the dressing rules of `ArmourRules`: the mage and the necromancer always wear a robe and a hood (the robe replaces chest, skirt and pants); the melee classes never wear a robe or a hood, except the assassin, who may wear the mage's hood; a melee skeleton always keeps a chest piece and leaves any other slot empty more often (`meleeBareChance`, 0.3) than a caster (`bareChance`, 0.1);
- a loadout that suits its class (`randomLoadout`), a subtle colour variation (`tintStrength`), the team's `eyeColor` and `team`;
- the upright gait for casters and the heavy gait for the rest (`gaitByClass`);
- a rise from the ground with a random delay (`riseFromGround`, `riseStagger`), the AI switching on when it has risen.

`Spawn()` and `Clear()` are also in the component's context menu; `spawned` lists the skeletons it created.

13. The arena demo

Two armies face each other on a small arena with pillars and walls. The panel raises 8 or 16 skeletons per side, starts and stops the battle, clears the field, and switches cloth, the death mode and the dissolving of corpses.

Input	Action
Left click	select a skeleton: its AI stops and it follows your orders
Right click	move the selected skeleton (hold Shift to run), or send it at an enemy
Ctrl + left click	strike any skeleton (hit reaction; pushes a ragdoll)
Alt + left click	kill a skeleton with a heavy blow
WASD / arrows, middle drag	pan the camera
Right drag, Q / E	rotate the camera
Mouse wheel	zoom

14. The extended animation browser

`UndeadLegion/Extended/Scenes/UndeadLegion_Extended_Showcase.unity` is the base pack's animation browser with the extended characters and a **PHYSICS** panel: cloth on or off, a physical hit, a ragdoll death, revive, rise from the ground and turn to dust. Click the skeleton to hit it where you click; Shift + click kills it into a ragdoll pushed from that point, and clicks on the fallen body push it around. Picking any clip, or Revive, stands a fallen skeleton back up. The browser shows one character at a time, so the component `ExtendedShowcase` removes its navigation and AI when it spawns; the `Hit_01` and `Stagger_01` clips are in the Reactions section, and armour pieces put on from the modules panel get cloth.

15. The player demo

`UndeadLegion/Extended/Scenes/UndeadLegion_Extended_Player.unity`: control one skeleton against waves of AI skeletons on the arena.

Input	Action
W A S D	walk forward, back and sideways on the locomotion blend tree (root motion, the clips' measured speeds)
Shift + W	run
Mouse	turn (the camera follows behind the shoulder); wheel zooms
Left click	attack with the moves of the attack modules attached to the weapon; each click plays the next move, a click late in an attack chains the next
Right button (hold)	raise the shield (a loadout with a shield): frontal hits are blocked
Esc	pause; the menu switches the class (the new skeleton replaces the old one where it stands), the weapon, and how many enemies fight at once

The controlled skeleton cannot die (`SkeletonHealth.invulnerable`) but shows every hit: the physical flinch, the hit clips, a camera kick and a red flash. Three components make it up, each usable on its own: `PlayerSkeleton` (on the character: input to the `SkeletonNavController`'s manual mode, attacks, block; the `SkeletonAI` stays on it, disabled, as the source of the attack modules), `PlayerCamera` (on the camera) and `PlayerDemo` (the scene: the player, the enemy waves from an `ArmySpawner`, the menu and the HUD). `SkeletonNavController.manual` with `ManualVelocity` (in the character's frame) is the hook for your own character controller.

16. Performance

- Ragdoll bodies are kinematic until death, so a living skeleton costs only its colliders.
- Cloth is the most expensive feature: keep `cullDistance` modest, or remove `SkeletonCloth` from background crowds.
- The rise and dissolve material copies are shared per source material and only used during an effect.
- Eyes are two unlit spheres; their optional light is off by default.
- For large crowds, set the agents' obstacle avoidance quality lower and consider disabling `SkeletonTwitch` and the hit reactions far from the camera.

17. Support

For questions or issues, email **vadymudod@hotmail.com**. Please include your Unity version, the edition (Extended) and a short description or screenshot of the problem.